



ITES
RENÉ DESCARTES

LICENCIATURA:

PROGRAMACIÓN Y WEB MASTER

MAESTRO:

MARIO ALONSO SEGOVIA GUTIERREZ

MATERIA:

PROYECTO DE UNA PLATAFORMA VIRTUAL

ALUMNO:

MICHAEL ROMÁN ALAVEZ COLLÍ

Introducción

Los patrones de arquitectura de software son formas organizadas de estructurar un sistema para facilitar su desarrollo, mantenimiento y crecimiento. Estos patrones permiten dividir responsabilidades, ordenar el código y mejorar la comunicación entre los componentes del sistema.

En el desarrollo de software moderno, elegir una arquitectura adecuada es muy importante, ya que influye directamente en la escalabilidad, el rendimiento, la seguridad, la mantenibilidad y los costos del proyecto. No todos los sistemas necesitan la misma arquitectura, por lo que es necesario analizar las características del proyecto antes de seleccionar una solución.

En este trabajo se comparan cinco patrones de arquitectura: MVC, arquitectura en capas, arquitectura monolítica, microservicios y cliente-servidor. Se identifican sus características principales, ventajas, desventajas y casos de uso recomendados.

Cuadro comparativo

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Objetivo principal	Separar la lógica, la vista y el control de la aplicación.	Dividir el sistema por niveles de responsabilidad.	Integrar todo el sistema en una sola aplicación.	Dividir el sistema en servicios independientes.	Separar las funciones entre clientes y un servidor central.
Estructura	Modelo, Vista y Controlador.	Presentación, lógica de negocio, datos y acceso a datos.	Una sola base de código con todos los módulos juntos.	Servicios pequeños conectados mediante APIs.	Clientes que solicitan información a un servidor.
Organización del código	Ordenado por responsabilidades.	Organizado por capas funcionales.	Todo se encuentra dentro del mismo proyecto.	Cada servicio tiene su propio código y lógica.	El cliente maneja la interfaz y el servidor procesa datos.
Escalabilidad	Media.	Media.	Baja o limitada.	Alta.	Media o alta, según el servidor.
Mantenimiento	Fácil si el proyecto está bien estructurado.	Fácil por la separación de capas.	Puede volverse difícil cuando el sistema crece.	Más sencillo por servicio, pero más complejo en general.	Depende de la comunicación entre cliente y servidor.
Complejidad	Media.	Media.	Baja al inicio.	Alta.	Media.
Facilidad de implementación	Relativamente fácil.	Fácil de entender y aplicar.	Muy fácil al inicio.	Difícil por la configuración y comunicación entre servicios.	Fácil para sistemas básicos.
Rendimiento	Bueno en aplicaciones web.	Bueno si las capas están bien diseñadas.	Bueno al inicio, pero puede disminuir al crecer.	Alto si está bien implementado.	Depende de la red y del servidor.
Reutilización	Buena reutilización de modelos y vistas.	Permite reutilizar componentes por capas.	Baja reutilización.	Alta reutilización de servicios.	Se pueden reutilizar clientes o servicios del servidor.
Casos de uso recomendados	Aplicaciones web, sistemas Laravel, plataformas administrativas.	Sistemas empresariales, escolares o administrativos.	Proyectos pequeños o prototipos.	Sistemas grandes, ecommerce, streaming o bancos.	Aplicaciones web, móviles o sistemas conectados a una base de datos.
Ventajas	Ordena el código, facilita mantenimiento y separa responsabilidades.	Estructura clara, fácil mantenimiento y buena organización.	Simple, rápido de desarrollar y económico al inicio.	Escalable, flexible y permite trabajar por equipos separados.	Centraliza datos y permite acceso desde varios dispositivos.
Desventajas	Puede ser confuso si no se separan bien las responsabilidades.	Puede generar demasiadas capas innecesarias.	Difícil de escalar y mantener cuando crece.	Requiere más infraestructura, monitoreo y conocimientos técnicos.	Depende de la conexión y puede saturar el servidor.

Respuestas de análisis

1. ¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?

Para una plataforma web desarrollada por un equipo pequeño, considero que el patrón más adecuado es MVC, porque permite organizar el sistema de forma clara sin hacerlo demasiado complejo. Además, muchos frameworks web como Laravel utilizan este patrón, lo cual facilita el desarrollo, mantenimiento y crecimiento del proyecto.

MVC permite separar la vista, la lógica y los datos, por lo que cada parte del sistema queda mejor organizada. Para un equipo pequeño es útil porque no requiere tanta infraestructura como los microservicios, pero sí ofrece una estructura profesional.

2. ¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?

Los microservicios ofrecen mayor escalabilidad, ya que cada servicio puede crecer de forma independiente. También permiten que diferentes equipos trabajen en distintas partes del sistema sin afectar toda la aplicación.

Otra ventaja es que si un servicio falla, no necesariamente se cae todo el sistema. En cambio, en una arquitectura monolítica, todos los módulos están unidos, por lo que un error puede afectar a toda la aplicación.

3. ¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?

MVC sigue siendo muy utilizado porque es fácil de entender, organiza bien el código y separa responsabilidades. La vista se encarga de mostrar la información, el modelo maneja los datos y el controlador coordina las acciones del usuario.

Además, muchos frameworks populares utilizan MVC, por ejemplo Laravel, CodeIgniter, Ruby on Rails y ASP.NET MVC. Esto hace que sea un patrón práctico y común en el desarrollo web.

4. ¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?

El principal riesgo es invertir más tiempo y recursos de los necesarios. Una arquitectura demasiado compleja puede hacer que el desarrollo sea más lento, difícil de entender y costoso de mantener.

También puede provocar confusión en el equipo, errores de configuración y problemas innecesarios. Por ejemplo, usar microservicios en un sistema pequeño podría ser excesivo, ya que requiere servidores, APIs, monitoreo y mayor administración.

5. ¿Crees que existe un patrón mejor que todos los demás? Justifica tu respuesta.

No existe un patrón mejor que todos los demás, porque cada arquitectura depende del tipo de proyecto, el tamaño del equipo, el presupuesto, la cantidad de usuarios y los objetivos del sistema.

Por ejemplo, una arquitectura monolítica puede ser buena para un proyecto pequeño, mientras que los microservicios pueden ser mejores para un sistema grande con muchos usuarios. Lo importante es elegir la arquitectura que mejor se adapte a las necesidades del proyecto.

Caso práctico

Sistema seleccionado: Plataforma educativa

Para desarrollar una plataforma educativa utilizaría el patrón MVC combinado con arquitectura en capas.

Esta decisión se debe a que una plataforma educativa necesita manejar usuarios, cursos, docentes, alumnos, tareas, calificaciones y contenidos. MVC ayudaría a organizar la aplicación separando la interfaz, la lógica y los datos. La arquitectura en capas permitiría dividir mejor las responsabilidades del sistema.

Justificación

Número de usuarios:

Una plataforma educativa puede tener muchos usuarios, como alumnos, docentes y administradores. MVC permite manejar estos roles de forma ordenada.

Escalabilidad:

Aunque al inicio puede ser un sistema mediano, puede crecer con el tiempo. La arquitectura en capas permite agregar nuevas funciones sin afectar todo el sistema.

Complejidad:

La plataforma tiene una complejidad media, ya que incluye gestión de usuarios, cursos, actividades y calificaciones. MVC es adecuado porque organiza cada módulo.

Mantenimiento:

El mantenimiento sería más sencillo, ya que el código estaría separado por responsabilidades. Esto facilita corregir errores o agregar nuevas funciones.

Costos:

MVC y arquitectura en capas no requieren tanta infraestructura como los microservicios, por lo que los costos serían más bajos.

Tamaño del equipo de desarrollo:

Para un equipo pequeño o mediano, MVC es una buena opción porque es fácil de implementar y entender. Además, permite trabajar de manera ordenada sin complicar demasiado el proyecto.

Conclusión

En esta actividad aprendí que los patrones de arquitectura de software son fundamentales para organizar correctamente un sistema. Cada patrón tiene características, ventajas y desventajas que deben analizarse antes de iniciar un proyecto.

El patrón que me pareció más interesante fue MVC, porque es muy utilizado en aplicaciones web y permite separar de forma clara la información, la interfaz y la lógica del sistema. Además, es práctico para proyectos desarrollados con frameworks como Laravel.

La arquitectura influye directamente en la calidad de un sistema, ya que afecta su mantenimiento, rendimiento, escalabilidad y facilidad de desarrollo. Una mala elección puede provocar problemas cuando el sistema crece o necesita cambios.

Por eso, es importante seleccionar adecuadamente la arquitectura desde el inicio del proyecto. No se trata de elegir la arquitectura más moderna o compleja, sino la que mejor se adapte a las necesidades reales del sistema, el equipo y los recursos disponibles.